

Strukturni Dizajn Paterni

1. Facade

Facade je obrazac dizajna koji služi za pojednostavljivanje komunikacije između više klasa u sistemu. Umjesto da jedna klasa direktno komunicira sa velikim brojem drugih klasa, uvodi se jedna facade klasa koja predstavlja glavnu pristupnu tačku određenom dijelu sistema.

U našem projektu Facade pattern smo koristili kroz klasu `RezervacijaFacade`. Razlog za to je što proces kreiranja rezervacije uključuje više različitih funkcionalnosti i klasa. Tokom rezervacije potrebno je provjeriti dostupnost parking mjesta, izračunati cijenu, izvršiti plaćanje, kreirati rezervaciju i poslati email obavijest korisniku.

Bez Facade patterna, `RezervacijaController` bi morao direktno komunicirati sa klasama `ParkingMjesto`, `Placanje`, `Rezervacija` i `EmailObavijest`, što bi dodatno zakomplikovalo kontroler i povećalo broj zavisnosti između klasa.

Zbog toga smo dodali klasu `RezervacijaFacade`, preko koje se obavlja kompletan proces rezervacije. Na taj način kontroler komunicira samo sa facade klasom, dok facade interno koordinira rad ostalih dijelova sistema. Ovo rješenje čini sistem preglednijim, organizovanijim i lakšim za održavanje.

2. Adapter

Adapter je obrazac dizajna koji omogućava povezivanje klasa ili servisa koji imaju različite interfejse.

U našem projektu Adapter pattern smo koristili u dijelu sistema koji je zadužen za online plaćanje rezervacija. Naš sistem koristi metode kao što su:

`izvrsiPlacanje()`
`pokreniPovratNovca()`

Međutim, vanjski servisi za plaćanje, kao što su Stripe i PayPal, koriste drugačije metode. Naprimjer, Stripe koristi metode:

`processPayment()`
`refundPayment()`

dok PayPal koristi:

payNow()
refund()

Kako naš sistem ne bi bio direktno vezan za konkretan servis za plaćanje, dodali smo interfejs **IPlatniServis** i adapter klase **StripeAdapter** i **PayPalAdapter**.

Adapteri služe kao posrednici između našeg sistema i eksternih API-ja za plaćanje. Oni prevode metode koje koristi naš sistem u metode koje razumiju konkretni servisi za plaćanje.

Na ovaj način možemo lako dodati novi servis za plaćanje bez izmjene postojeće logike sistema. Dovoljno je samo napraviti novi adapter koji implementira isti interfejs.

3. Decorator

Decorator je obrazac dizajna koji omogućava dodavanje novih funkcionalnosti objektima bez izmjene njihove osnovne klase.

U našem projektu Decorator pattern bi se mogao koristiti za dodavanje dodatnih opcija rezervaciji. Naprimjer, korisnik bi mogao izabrati natkriveno parking mjesto, VIP mjesto, punjenje električnog vozila ili dodatne obavijesti. Umjesto da sve te opcije direktno dodajemo u klasu *Rezervacija*, mogli bismo koristiti decorator klase koje bi proširivale osnovnu rezervaciju dodatnim funkcionalnostima i dodatnim troškovima. Na taj način osnovna klasa *Rezervacija* ostaje jednostavna, dok se dodatne opcije mogu dodavati po potrebi.

4. Bridge

Bridge obrazac omogućava odvajanje apstrakcije od implementacije kako bi se oba dijela mogla nezavisno proširivati.

U našem projektu ovaj pattern bi se mogao koristiti u sistemu obavijesti. Trenutno sistem koristi klasu *EmailObavijest*, ali bi se u budućnosti mogli dodati i drugi načini obavješćavanja korisnika, kao što su SMS poruke, push notifikacije ili mobilne obavijesti. Korištenjem Bridge patterna mogli bismo odvojiti tip obavijesti od načina slanja obavijesti. To znači da bi ista obavijest mogla biti poslana putem emaila, SMS-a ili push notifikacije bez izmjene osnovne logike sistema.

5. Composite

Composite obrazac omogućava organizovanje objekata u hijerarhijske strukture, tako da se pojedinačni objekti i grupe objekata mogu koristiti na isti način. U ovom projektu ovaj pattern bi se mogao koristiti za organizaciju parking prostora. Pored parking mjesta, sistem bi mogao sadržavati parking sekcije, redove, spratove ili zone. Naprimjer, jedna parking zona može sadržavati više sekcija, a svaka sekcija više parking mjesta. Composite pattern bi omogućio da se i cijela zona i pojedinačno parking mjesto tretiraju na sličan način. To bi olakšalo pregled i upravljanje slobodnim parking mjestima unutar sistema.

6. Proxy

Proxy obrazac služi za kontrolu pristupa određenim objektima ili funkcionalnostima sistema. Proxy pattern bi se mogao koristiti za provjeru prava pristupa korisnika prije izmjene ili otkazivanja rezervacije.

Naprimjer, sistem bi mogao provjeravati:

- da li je korisnik prijavljen,
- da li je nalog aktivan,
- da li korisnik ima odgovarajuću ulogu,
- da li korisnik ima pravo pristupa određenoj rezervaciji.

Na taj način povećava se sigurnost sistema i zaštita podataka korisnika.

7. Flyweight

Flyweight obrazac omogućava efikasnije korištenje memorije dijeljenjem zajedničkih podataka između više objekata. U našem projektu ovaj pattern bi se mogao koristiti za podatke koji se često ponavljaju, kao što su tipovi parking mjesta, statusi rezervacija, statusi plaćanja ili podaci o parking zonama. Umjesto kreiranja velikog broja istih objekata, zajednički podaci bi se dijelili između više dijelova sistema kako bi se smanjilo korištenje memorije.

Ipak, prema zahtjevima zadatka, Flyweight pattern nije bilo potrebno implementirati u klasni dijagram.